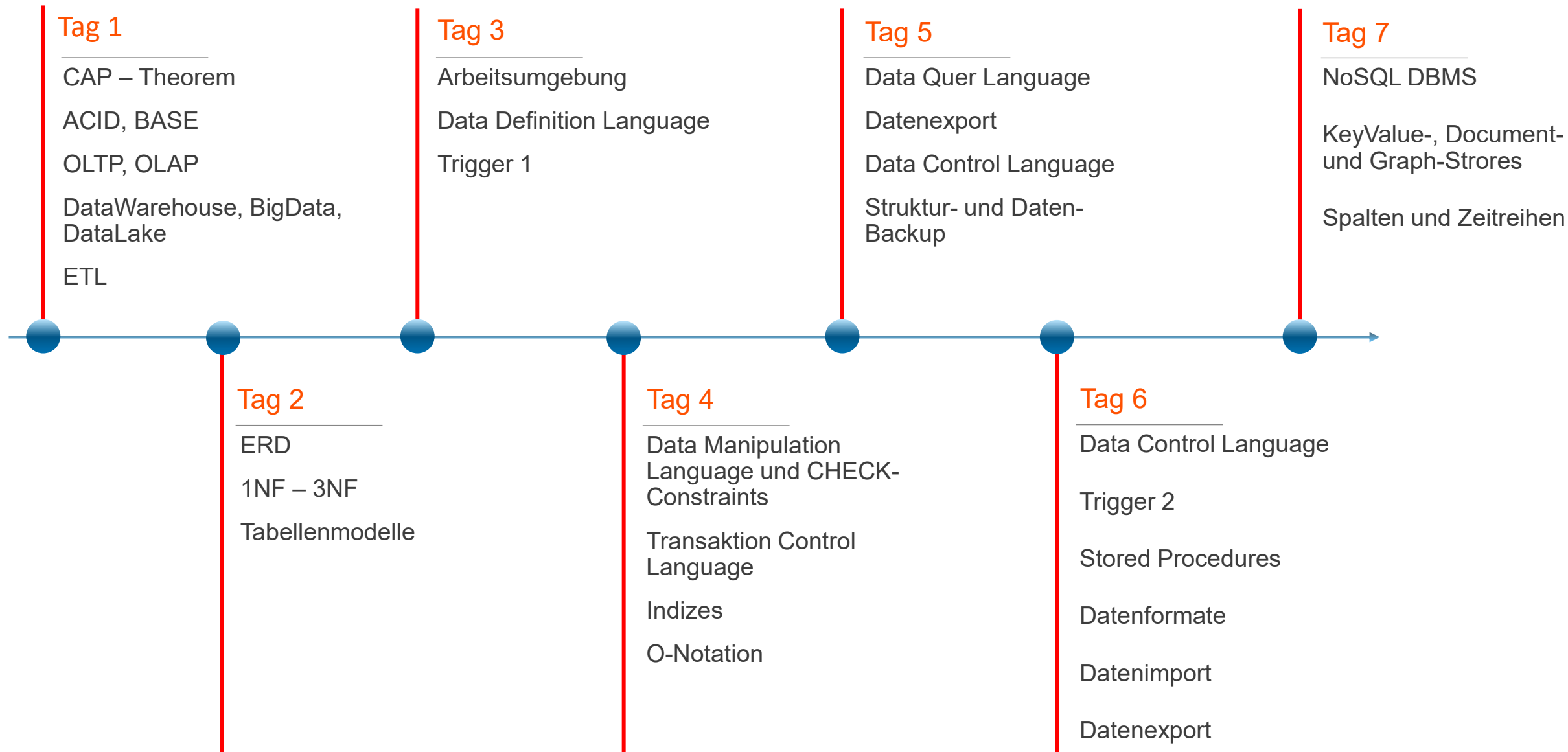
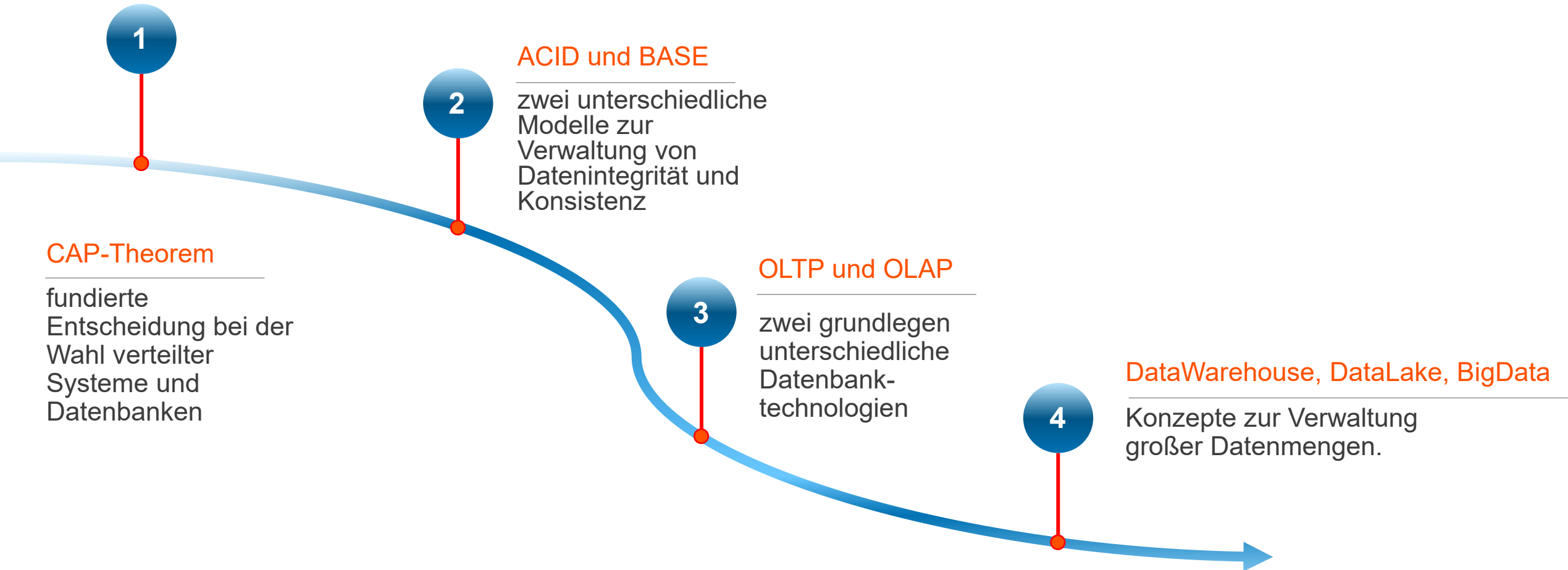


DB2



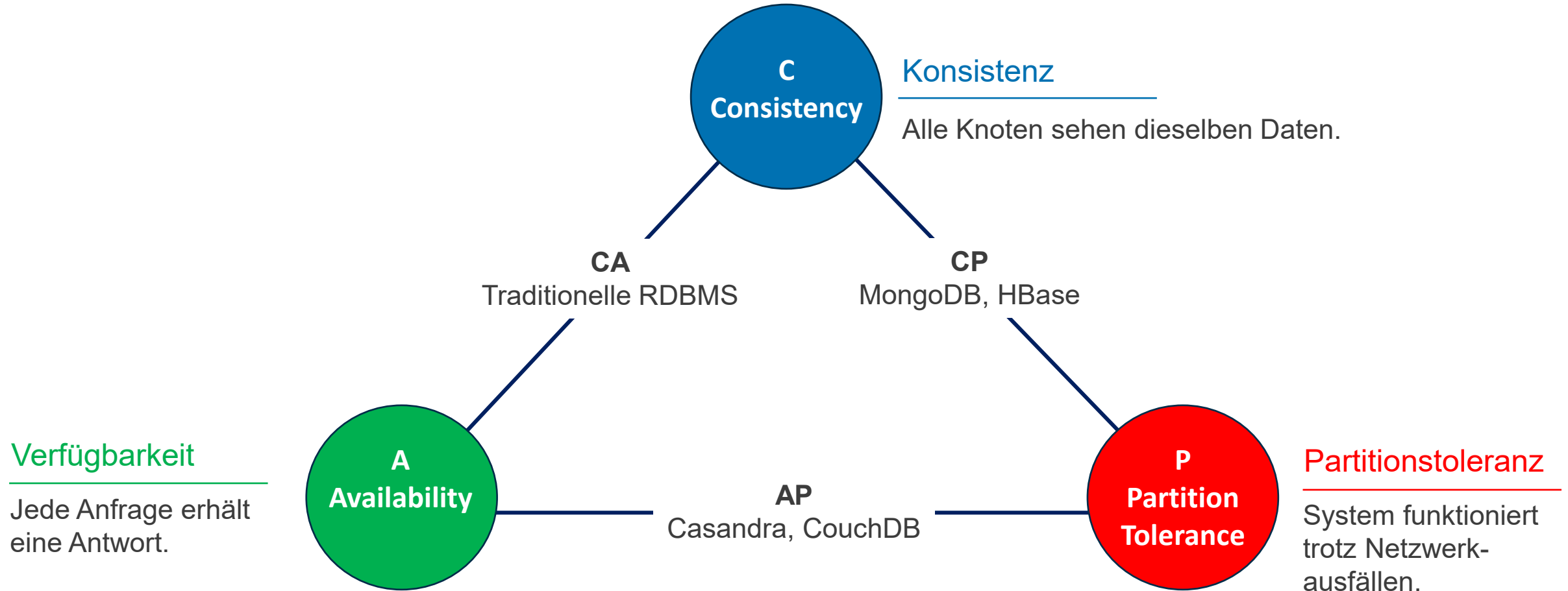




CAP-Theorem

Eine fundierte Entscheidung bei der Wahl verteilter Datenbanken-Systeme

Ein verteiltes System kann maximal zwei der drei Eigenschaften garantieren



Consistency (Konsistenz)

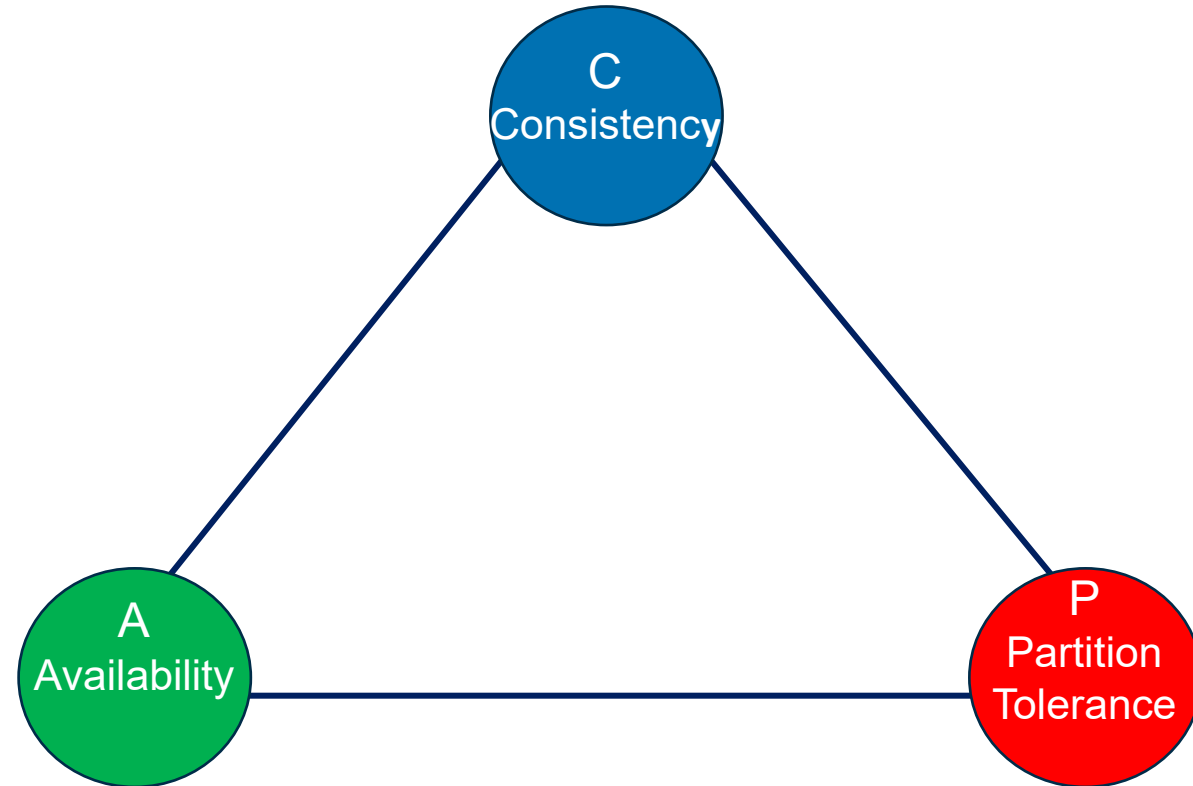
Alle Knoten sehen zur gleichen Zeit die gleichen Daten. Jeder Lesevorgang erhält die aktuellste geschriebene Information.

Availability (Verfügbarkeit)

Jede Anfrage erhält eine Antwort – auch wenn einzelne Knoten ausfallen. Das System ist immer erreichbar.

Partition Toleranz (Partitiontoleranz)

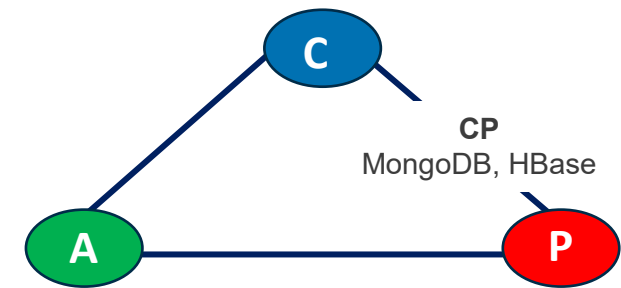
Das System funktioniert weiter, selbst wenn die Netzwerkverbindung zwischen Teilen unterbrochen wird.



Consistency + Partition Tolerance (CP):

Wenn absolute Datenkorrektheit wichtiger ist als ständige Verfügbarkeit.

Merke: CP-Systeme opfern Verfügbarkeit für Konsistenz.



Banküberweisungen

Bei Überweisungen zwischen Konten muss garantiert sein, dass Geld nicht doppelt abgebucht oder verloren geht. Wenn ein Teil des Systems ausfällt, werden Transaktionen gestoppt, bis die Konsistenz wiederhergestellt ist. Lieber kurzzeitig keine Überweisung möglich, als falscher Kontostand.

Reservierungssysteme (Flüge, Hotels)

Sitzplätze oder Hotelzimmer dürfen nicht doppelt gebucht werden. Das System muss garantieren, dass jede Ressource nur einmal vergeben wird. Bei Ausfällen werden keine Buchungen akzeptiert, um Doppelbuchungen zu vermeiden.

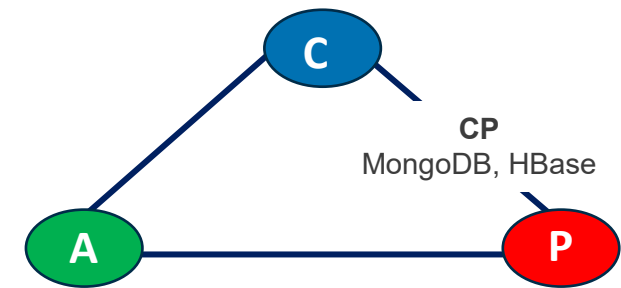
Medizinische Patientenakten

Diagnosen, Medikationen und Behandlungshistorien müssen immer korrekt und aktuell sein. Falsche oder veraltete Informationen könnten lebensbedrohlich sein. Das System wird lieber temporär unverfügbar, als inkonsistente Daten anzuzeigen.

Consistency + Partition Tolerance (CP):

Wenn absolute Datenkorrektheit wichtiger ist als ständige Verfügbarkeit ist.

Merke: CP-Systeme opfern Verfügbarkeit für Konsistenz.



MongoDB

MongoDB ist eine dokumentenorientierte NoSQL-Datenbank, die bei Netzwerkpartitionen die Konsistenz gewährleistet. Wenn ein Knoten vom Cluster getrennt wird, stoppt er Schreibvorgänge, bis die Verbindung wiederhergestellt ist. Nur der Primary-Knoten akzeptiert Schreibvorgänge.

HBase

HBase basiert auf dem BigTable-Design von Google und garantiert starke Konsistenz durch ein Master-Slave-Modell. Bei Verbindungsproblemen werden Server, die nicht erreichbar sind, als offline markiert, wodurch ein Schreibvorgang blockiert wird, bis das Problem behoben ist.

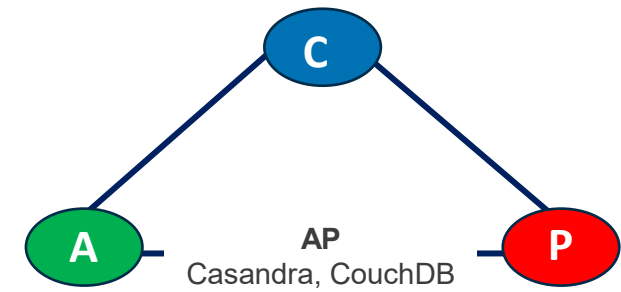
Redis (im Standard-Modus)

Redis bietet im Cluster-Modus mit Replikation starke Konsistenz. Bei Verbindungsproblemen können Slaves die Verbindung zum Master verlieren und werden dann schreibgeschützt, um inkonsistente Daten zu vermeiden.

Availability + Partition Tolerance (AP):

Wenn das System immer erreichbar sein muss, auch wenn Daten kurzzeitig inkonsistent sind.

Merke: AP-Systeme akzeptieren temporäre Inkonsistenzen für hohe Verfügbarkeit.



Social Media Feeds (Facebook, Twitter/X)

Nutzer müssen jederzeit Beiträge lesen und posten können. Es ist akzeptabel, wenn "Likes" oder Kommentarzeilen für kurze Zeit unterschiedlich angezeigt werden. Hauptsache die Plattform bleibt erreichbar, die Synchronisation erfolgt später.

DNS-System

Das Domain Name System muss weltweit verfügbar sein. Wenn ein DNS-Server kurzzeitig veraltete IP-Adressen liefert, ist das tolerierbar. Wichtiger ist, dass Websites überhaupt aufrufbar bleiben, auch wenn die Info einige Minuten alt ist.

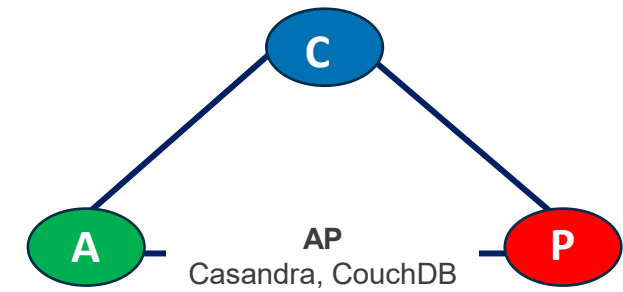
IoT-Sensor-Datensammlung

Tausende Sensoren senden ständig Temperaturen, Luftfeuchtigkeit etc. Wenn einige Datenpunkte verzögert ankommen oder temporär fehlen, ist das verschmerzbar. Wichtiger ist, dass das System weiterläuft und Daten sammelt.

Availability + Partition Tolerance (AP):

Wenn das System immer erreichbar sein muss, auch wenn Daten kurzzeitig inkonsistent sind.

Merke: AP-Systeme akzeptieren temporäre Inkonsistenzen für hohe Verfügbarkeit.



Apache Cassandra

Cassandra ist eine spaltenorientierte NoSQL-Datenbank, die für hohe Verfügbarkeit konzipiert ist. Jeder Knoten kann Lese- und Schreib Anfragen bearbeiten, selbst bei Verbindungsproblemen. Inkonsistenzen werden später durch Anti-Entropy-Reparatur und Hinted Handoff aufgelöst.

CouchDB

CouchDB nutzt ein Multi-Master-Replikationsmodell, bei dem alle Knoten gleichzeitig Schreibvorgänge akzeptieren können. Konflikte werden durch die Versionierung erkannt und können später manuell oder automatisch aufgelöst werden.

Amazon DynamoDB

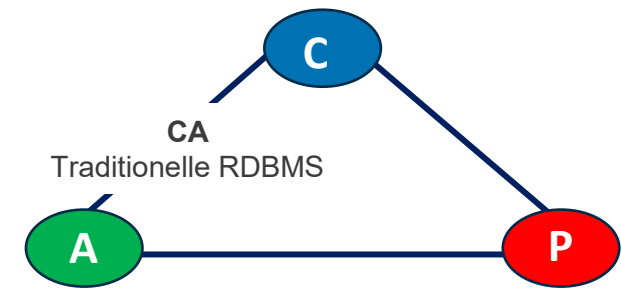
DynamoDB bietet konfigurierbare Konsistenzlevel. Im „eventual consistency“-Modus bleibt das System auch bei Datenpartitionierung verfügbar und synchronisiert Daten asynchron. Dies ermöglicht extrem hohe Verfügbarkeit z.B. für Amazons E-Commerce-Plattform.

Praktische Beispiele – Anwendungsfälle

Consistency + Availability (CA):

In Single-Server-Umgebungen oder wenn Netzwerkpartitionen praktisch ausgeschlossen sind.

Merke: CA-Systeme sind hauptsächlich Single-Node-Datenbanken, da echte Partitions-toleranz in verteilten Systemen unvermeidbar ist.



Interne Firmendatenbank (lokales Netzwerk)

Eine Personaldatenbank, die nur im Firmen-LAN läuft, bietet Konsistenz und Verfügbarkeit. Netzwerkausfälle im LAN sind extrem selten. Solange das lokale Netzwerk stabil ist, funktioniert das System perfekt.

Einzelhandel-Kassensystem (Stand-alone)

Eine Kasse ohne Netzwerk-Verbindung speichert Verkäufe lokal und konsistent. Solange der Kassensystemcomputer läuft, funktioniert alles. Synchronisation mit der Zentrale erfolgt später bei Verbindung.

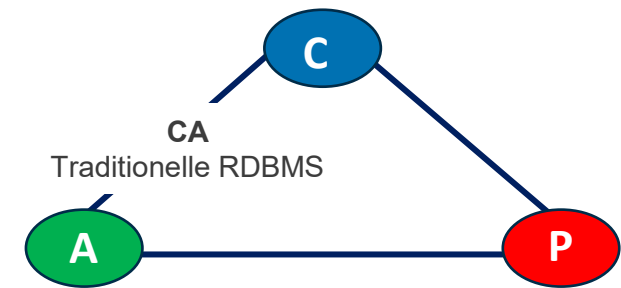
Entwicklungs- und Testumgebungen

Lokale Entwickler-Datenbanken auf dem eigenen Rechner brauchen keine Partitionstoleranz. Sie sind verfügbar und konsistent für den einzelnen Entwickler. Perfekt für isolierte Entwicklung.

Consistency + Availability (CA):

In Single-Server-Umgebungen oder wenn Netzwerkpartitionen praktisch ausgeschlossen sind.

Merke: CA-Systeme sind hauptsächlich Single-Node-Datenbanken, da echte Partitions-toleranz in verteilten Systemen unvermeidbar ist.



PostgreSQL (Single-Node)

Eine einzelne PostgreSQL-Instanz garantiert ACID-Eigenschaften und ist hochverfügbar, solange keine Netzwerkpartition auftritt. Bei Ausfall der Netzwerkverbindung kann das System nicht mehr funktionieren, da es nicht für verteilte Szenarien ausgelegt ist.

mariaDB (Single-Node)

Eine standalone mariaDB-Datenbank bietet vollständige Konsistenz und Verfügbarkeit innerhalb eines einzelnen Servers. Netzwerkpartitionen sind nicht vorgesehen – bei Netzwerkproblemen wird das System unzugänglich.

Microsoft SQL Server (Single-Instance)

MS SQL Server in einer nicht-verteilten Konfiguration garantiert starke Konsistenz durch ACID-Transaktionen und hohe Verfügbarkeit auf dem lokalen System. Partitionstoleranz ist nicht Teil des Designs.

Was kann zusammengefasst zu CAP-Theorem festgestellt werden?

CP-Systeme (Consistency + Partition Tolerance)

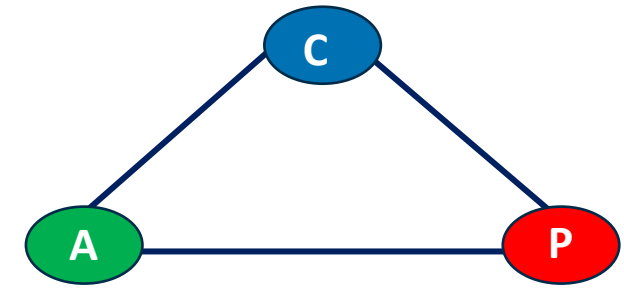
- priorisieren Konsistenz und Partitionstoleranz, opfern dafür Verfügbarkeit.

AP-Systeme (Availability + Partition Tolerance)

- priorisieren Verfügbarkeit und Partitionstoleranz, akzeptieren dafür temporäre Inkonsistenzen

CA-Systeme (Consistency + Availability)

- priorisieren Konsistenz und Verfügbarkeit, können Netzwerkpartitionen nicht tolerieren



Wichtige Anmerkung

In der Praxis sind reine CA-Systeme in verteilten Umgebungen nicht möglich, da Netzwerkpartitionen in realen Netzwerken unvermeidbar sind. Die aufgeführten CA-Beispiele sind daher hauptsächlich Anwendungen auf einzelnen Geräten oder in sehr stabilen lokalen Netzwerken. Sobald eine Anwendung über mehrere Rechenzentren oder das Internet verteilt ist, muss man sich zwischen CP und AP entscheiden.

2

ACID und BASE

Modelle zur Verwaltung von Datenintegrität und Konsistenz.

ACID und BASE

ACID und BASE sind zwei grundlegend verschiedene Philosophien für den Umgang mit Transaktionen in Datenbanksystemen.

- ACID setzt auf starke Konsistenzgarantien.
- BASE bevorzugt Verfügbarkeit und Fehlertoleranz.

ACID – Traditionelle relationale Datenbanken

Atomicity (Atomarität)

Transaktionen sind "alles oder nichts". Entweder werden alle Änderungen durchgeführt oder gar keine. Bei einem Fehler wird alles zurückgerollt.

Consistency (Konsistenz)

Jede Transaktion überführt die Datenbank von einem gültigen Zustand in einen anderen gültigen Zustand. Alle Integritätsbedingungen bleiben erhalten.

Isolation (Isolation)

Parallele Transaktionen beeinflussen sich nicht gegenseitig. Jede Transaktion läuft so, als wäre sie die einzige im System.

Durability (Dauerhaftigkeit)

Einmal bestätigte Transaktionen bleiben dauerhaft gespeichert, selbst bei Systemabstürzen oder Stromausfällen.

BASE – NoSQL und verteilte Systeme

Basically Available (Grundsätzlich verfügbar)

Das System garantiert Verfügbarkeit. Jede Anfrage erhält eine Antwort, auch wenn diese möglicherweise nicht die aktuellsten Daten enthält.

Soft State (Weicher Zustand)

Der Systemzustand kann sich über Zeit ändern, auch ohne neue Eingaben, da Daten zwischen Knoten repliziert werden. Keine strikte Konsistenz zu jedem Zeitpunkt.

Eventually Consistent (Letztendliche Konsistenz)

Das System wird irgendwann konsistent, aber nicht sofort. Nach einer Weile ohne Änderungen erreichen alle Knoten denselben Zustand.

ACID – Vorteile

Garantierte Datenkonsistenz

Daten sind immer in einem gültigen, korrekten Zustand.

Einfaches Programmiermodell

Entwickler müssen sich keine Gedanken über Inkonsistenzen machen.

Verlässliche Transaktionen

Klare "committed" oder "rolled back" Zustände.

Ideal für kritische Daten

Perfekt für Finanztransaktionen, Buchungen, medizinische Daten.

Starke Integritätsgarantien

Foreign Keys, Constraints werden strikt durchgesetzt.

Vorhersagbares Verhalten

Keine überraschenden Dateninkonsistenzen.

BASE – Vorteile

Hervorragende Skalierbarkeit

Einfaches horizontales Wachstum durch Hinzufügen von Servern.

Hohe Verfügbarkeit

System bleibt auch bei Ausfällen einzelner Knoten verfügbar.

Bessere Performance

Keine Locking-Mechanismen, schnellere Schreib- und Lesevorgänge.

Niedrige Latenz

Lokale Operationen ohne Warten auf globale Bestätigung.

Kosteneffizient

Kann auf günstiger Commodity-Hardware laufen.

Partitionstoleranz

Funktioniert auch bei Netzwerkproblemen weiter.

ACID – Nachteile

Schlechtere Skalierbarkeit

Horizontale Skalierung ist schwierig und teuer.

Performance-Overhead

Locking und Koordination verlangsamen das System.

Verfügbarkeitsprobleme

Bei Partitionen kann das System blockieren.

Höhere Latenz

Transaktionen müssen auf Bestätigung aller Knoten warten.

Single Point of Failure

Bei klassischen Setups kann ein Ausfall das gesamte System lahmlegen.

Teure Hardware

Oft werden leistungsstarke einzelne Server benötigt.

BASE – Nachteile

Komplexes Programmiermodell

Entwickler müssen mit eventueller Konsistenz umgehen.

Temporäre Inkonsistenzen

Verschiedene Knoten können unterschiedliche Daten liefern.

Konfliktauflösung nötig

Was passiert bei gleichzeitigen widersprüchlichen Updates?

Schwierige Fehlersuche:

Race Conditions und Timing-Probleme sind schwer zu debuggen.

Keine Transaktionsgarantien

Multi-Record-Updates können teilweise fehlschlagen.

Nicht für alle Anwendungen

Ungeeignet für Systeme, die strikte Konsistenz benötigen.

ACID – Typische Anwendungsfälle

Bankensysteme

Überweisungen müssen atomar sein. Geld darf nicht verloren gehen oder doppelt gebucht werden.
PostgreSQL, Oracle DB

Buchungssysteme

Flug- und Hotelbuchungen dürfen nicht doppelt vergeben werden. Strikte Konsistenz ist wichtig.
MS SQL Server

Inventarverwaltung

Lagerstände müssen exakt sein, um Überverkäufe zu vermeiden. Konsistenz ist wichtiger als Performance.
PostgreSQL

BASE – Typische Anwendungsfälle

Social Media Feeds

Ein paar Sekunden Verzögerung bei Likes sind akzeptabel. Wichtiger ist, dass die App immer funktioniert.
Cassandra, DynamoDB

Content Management

Blog-Posts und Kommentare müssen nicht sofort überall sichtbar sein. Wichtiger ist globale Verfügbarkeit.
Couchbase, Riak

IoT-Sensordaten

Tausende Sensoren senden kontinuierlich Daten. Hoher Durchsatz wichtiger als sofortige Konsistenz.
InfluxDB, TimescaleDB

ACID – Typische Anwendungsfälle (Fortsetzung)

E-Commerce Checkout

Bestellungen und Zahlungen müssen zuverlässig verarbeitet werden. Keine Doppelbuchungen oder verlorene Bestellungen.
MySQL, PostgreSQL

Gesundheitsdaten

Patientenakten und Medikamentenlisten müssen absolut korrekt sein. Keine veralteten oder inkonsistenten Informationen.
Oracle, DB2

BASE – Typische Anwendungsfälle (Fortsetzung)

Analytics & Logging

Millionen Events pro Sekunde. Exakte Konsistenz ist weniger wichtig als Durchsatz und Verfügbarkeit.
MongoDB, Elasticsearch

Shopping Carts BASE

Kunden sollen immer Artikel hinzufügen können. Warenkörbe können später synchronisiert werden.
DynamoDB, Redis

Wenn ein System mehr Last bewältigen muss – mehr Nutzer, mehr Anfragen, mehr Daten – Scale **Up** oder Scale **Out**.

Vertikale Skalierung (Scale Up)

Bei der vertikalen Skalierung wird ein einzelner Server stärker gemacht: mehr CPU-Kerne, mehr RAM, schnellere Festplatten. Das System wächst also „nach oben“.

Beispiel: Ein Datenbankserver hat 16 GB RAM und kommt an seine Grenzen. Man rüstet ihn auf 64 GB RAM auf.

Vorteile:

- Einfach umzusetzen – keine Änderungen an der Software nötig
- Kein Aufwand für Lastverteilung

Nachteile:

- Hat eine physische Obergrenze (man kann Hardware nicht unendlich aufrüsten)
- Teuer bei sehr leistungsstarker Hardware
- Single Point of Failure: Fällt die Maschine aus, fällt alles aus.

Horizontale Skalierung (Scale Out)

Bei der horizontalen Skalierung werden mehr Server hinzugefügt, die die Last gemeinsam tragen. Das System wächst „nach außen“.

Beispiel: Statt eines großen Servers betreibt man zehn kleinere, auf die ein Load Balancer die Anfragen verteilt.

Vorteile:

- Theoretisch unbegrenzt skalierbar
- Hohe Ausfallsicherheit – fällt ein Server aus, übernehmen die anderen
- Oft kostengünstiger mit Standard-Hardware

Nachteile:

- Höhere Komplexität (Lastverteilung, Datenkonsistenz, Netzwerk)
- Software muss oft angepasst werden (z. B. kein lokaler Session-Speicher)

Horizontale und vertikale Skalierung

Die Analogie: Stell dir vor, du musst mehr Bücher transportieren:

Vertikal: Du kaufst einen größeren LKW.

Horizontal: Du nimmst mehrere kleinere Transporter.

Szenario

Relationale Datenbanken

Webserver / APIs

Kurzfristig, schnell nötig

Langfristig, Cloud-nativ

Empfehlung

Oft vertikal (Konsistenz leichter)

Horizontal (zustandslos, einfach zu vervielfältigen)

Vertikal (weniger Aufwand)

Horizontal (flexibler, resilienter)

Anmerkung

In der Praxis kombinieren moderne Systeme beide Ansätze – man skaliert horizontal mit mehreren Knoten, die selbst jeweils ausreichend dimensioniert sind.

Was kann zusammengefasst zu ACID und BASE festgestellt werden?

- In Datenbanksystemen sind **ACID** und **BASE** zwei grundlegend verschiedene Philosophien für den Umgang mit Transaktionen.
- Während **ACID** auf starke Konsistenzgarantien setzt, bevorzugt **BASE** Verfügbarkeit und Fehlertoleranz.
- Die Wahl zwischen ACID und BASE hängt von den Anforderungen ab:
 - **ACID** ist ideal für Systeme, wo Datenkorrektheit oberste Priorität hat (Finanzen, Gesundheit, Buchungen).
 - **BASE** eignet sich für Systeme, die hohe Verfügbarkeit und Skalierbarkeit benötigen und temporäre Inkonsistenzen tolerieren können (Social Media, Analytics, Content Delivery).
- Moderne Systeme kombinieren oft beide Ansätze für unterschiedliche Datentypen.

3

OLTP und OLAP

Zwei verschiedene Welten der Datenverarbeitung im Vergleich

OLTP und OLAP

Was ist der Unterschied?

OLTP und OLAP sind zwei grundlegend verschiedene Ansätze für den Umgang mit Datenbanken.

- OLTP ist für das tägliche Geschäft gemacht – schnelle Transaktionen wie Bestellungen oder Buchungen.
- OLAP ist für Analysen gedacht – komplexe Auswertungen über große Datenmengen hinweg.

OLTP - Online Transaction Processing

Das operative Tagesgeschäft

OLTP-Systeme verarbeiten viele kleine, schnelle Transaktionen. Denk an einen Kassenvorgang im Supermarkt, eine Geldüberweisung oder eine Hotelbuchung – alles muss sofort passieren und zuverlässig sein.

- Viele kleine Transaktionen pro Sekunde
- Sehr schnelle Antwortzeiten (Millisekunden)
- Aktuelle Daten in nahezu Echtzeit
- Viele gleichzeitige Nutzer (hunderte bis tausende)
- Schreiben, Lesen, Ändern einzelner Datensätze

OLAP - Online Analytical Processing

Analyse für Entscheidungen

OLAP-Systeme werten riesige Datenmengen aus, um Trends zu erkennen und Berichte zu erstellen. Wie viel haben wir letztes Jahr verkauft? Welche Produkte laufen in welchen Regionen am besten? Das alles sind OLAP-Fragen.

- Wenige komplexe Abfragen über große Datenmengen
- Längere Antwortzeiten (Sekunden bis Minuten)
- Historische Daten zur Analyse
- Wenige gleichzeitige Nutzer (Analysten, Manager)
- Lesen und Aggregieren von Millionen Datensätzen

OLTP = Supermarktkasse

Die Kassiererin scannt jeden Artikel einzeln ein, der Preis wird sofort berechnet, der Lagerbestand wird aktualisiert. Jeder Vorgang dauert nur Sekunden. Hunderte Kassen arbeiten gleichzeitig.

Schnell, präzise, viele kleine Aktionen.

OLAP = Geschäftsführung

Am Monatsende sitzt der Geschäftsführer im Büro und analysiert: "Wie haben sich unsere Verkäufe entwickelt? Welche Filialen sind am profitabelsten?" Diese Auswertung dauert länger, schaut aber auf Millionen von Kassenvorgängen.

Wenige Nutzer, große Datenmengen, strategische Erkenntnisse.

Merkmal	OLTP	OLAP
Zweck	Tagesgeschäft, operative Prozesse	Datenanalyse, Business Intelligence
Datenquelle	Aktuelle, operative Daten	Historische Daten aus mehreren Quellen
Typische Operationen	INSERT, UPDATE, DELETE, SELECT einzelner Zeilen	SELECT mit GROUP BY, SUM, AVG über Millionen Zeilen
Antwortzeit	Millisekunden bis 1-2 Sekunden	Sekunden bis Minuten (manchmal Stunden)
Datenvolumen pro Abfrage	Klein (einzelne Zeilen oder wenige Zeilen)	Groß (Millionen bis Milliarden Zeilen)
Gleichzeitige Nutzer	Hunderte bis Tausende	Wenige bis einige Dutzend

Merkmal	OLTP	OLAP
Datenstruktur	Hochnormalisiert (3NF), viele Tabellen mit JOINS	Denormalisiert (Star/Snowflake), weniger JOINS
Datenbankgröße	GB bis TB	TB bis PB
Backup-Häufigkeit	Häufig (stündlich, täglich)	Seltener (wöchentlich, monatlich)
Optimiert für	Schreibvorgänge (Write-heavy)	Lesevorgänge (Read-heavy)
Indizierung	Wenige Indizes (zu viele verlangsamen Writes)	Viele Indizes, Aggregate, vorberechnete Views
Beispiel-Query	<code>UPDATE orders SET status='shipped' WHERE order_id=12345;</code>	<code>SELECT region, SUM(revenue) FROM sales GROUP BY region;</code>

Online-Banking **OLTP**

Du überweist Geld von deinem Konto. Die Transaktion muss sofort durchgeführt werden, dein Kontostand muss stimmen, und tausende andere Kunden machen gleichzeitig Überweisungen.

Flugbuchungssystem **OLTP**

Ein Passagier bucht einen Flug. Der Sitzplatz muss sofort reserviert werden, keine Doppelbuchungen erlaubt. Tausende Buchungen passieren gleichzeitig weltweit.

E-Commerce Checkout **OLTP**

Ein Kunde legt Artikel in den Warenkorb und bezahlt. Der Lagerbestand wird sofort aktualisiert, die Bestellung gespeichert. Jede Sekunde zählt für die User Experience.

Jahresabschluss-Analyse **OLAP**

Die Bank wertet alle Transaktionen des Jahres aus: Welche Produkte sind profitabel? In welchen Regionen wachsen wir? Diese Analysen laufen über Millionen Transaktionen.

Airline Revenue Management **OLAP**

Die Airline analysiert: "Welche Routen sind am profitabelsten?", "Wie war die Auslastung im letzten Jahr?", "Welche Preisstrategien funktionieren?" Daten aus Jahren werden ausgewertet.

Verkaufs-Dashboard **OLAP**

Das Management schaut auf ein Dashboard: "Top 10 Produkte dieses Quartals", "Umsatzentwicklung nach Region", "Customer Lifetime Value". Aggregierte Einblicke über alle Bestellungen.

Was kann zusammengefasst zu OLTP und OLAP festgestellt werden?

- Beide Systeme ergänzen sich:
 - **OLTP** sammelt die Daten; Geeignet für schnelle, zuverlässige Transaktionen im Tagesgeschäft – wie ein Kassierer, der Artikel scannt.
 - **OLAP** wertet sie aus; Geeignet für komplexe Analysen über große Datenmengen – wie ein Manager, der Verkaufszahlen analysiert.
- In der Praxis werden oft beide verwendet:
 - **OLTP**-Datenbanken für den Betrieb, und die Daten werden regelmäßig in ein **OLAP**-System (Data Warehouse) kopiert, wo Analysten sie auswerten können, ohne das operative Geschäft zu stören.

4

Data Warehouse, Big Data und Data Lake

Drei Konzepte für moderne Datenverwaltung

Data Warehouse, Big Data und Data Lake

Worum geht's?

In der digitalen Welt entstehen ständig riesige Datenmengen. Aber wie organisiert und speichert man sie am besten?

- Data Warehouse ist wie eine gut sortierte Bibliothek für strukturierte Geschäftsdaten.
- Big Data beschreibt die Herausforderung, mit enormen, schnell wachsenden Datenmengen umzugehen.
- Data Lake ist wie ein großer Datenspeicher, der alles aufnimmt – egal in welchem Format.

Data Warehouse

Ein Data Warehouse ist eine zentrale, gut organisierte Datenbank für Geschäftsanalysen. Die Daten sind strukturiert, bereinigt und optimiert für schnelle Abfragen und Berichte.

Big Data

Big Data ist kein Produkt, sondern ein Konzept für extrem große, komplexe Datenmengen, die mit traditionellen Methoden nicht mehr verarbeitet werden können. Definiert durch die „5 V's" (oder mehr).

Data Lake

Ein Data Lake ist ein riesiger Speicher für Rohdaten in beliebigem Format. Alles wird in Originalform gespeichert, die Struktur kommt erst beim Auslesen.

Data Warehouse

- Strukturierte Daten in Tabellen
- Schema-on-Write: Struktur wird beim Speichern festgelegt
- Nur relevante Daten für Business Intelligence
- Hohe Datenqualität durch ETL-Prozesse
- Teuer aber schnell

Big Data

- Volume: Riesige Datenmengen (TB bis PB)
- Velocity: Daten entstehen schnell (Echtzeit)
- Variety: Verschiedene Datentypen & Formate
- Veracity: Unterschiedliche Datenqualität
- Value: Wertvolle Erkenntnisse gewinnen
- Benötigt spezielle Tools: Hadoop, Spark, etc.

Data Lake

- Rohdaten in Originalformat
- Schema-on-Read: Struktur beim Lesen festlegen
- Alle Datentypen: strukturiert, semi-strukturiert, unstrukturiert
- Flexibel: Speichern jetzt, analysieren später
- Günstig: Cloud-Storage, z.B. Amazon S3
- Risiko: Kann zum "Data Swamp,, (Datensumpf) werden.

Data Warehouse

1980er – Data Warehouse

Erste Data Warehouses entstehen für Business Intelligence

Data Warehouse = Bibliothek

Alle Bücher sind sortiert, katalogisiert und an ihrem festen Platz. Du findest schnell, was du suchst. Aber: Nur Bücher, die den Standards entsprechen, kommen rein. Teuer zu betreiben.

Big Data

2000er – Big Data

Begriff wird populär, Hadoop & verteilte Systeme entstehen

Big Data = Informationsflut

Stell dir vor, jeden Tag kommen LKW-Ladungen neuer Bücher, Zeitungen, Videos, Audiodateien. Normale Bibliotheken können das nicht bewältigen – du brauchst neue Methoden und Werkzeuge.

Data Lake

2010er – Data Lake

Cloud-Speicher macht günstige, flexible Datenspeicherung möglich.

Data Lake = Lagerhalle

Alles wird einfach reingeworfen – Bücher, Zeitungen, Notizen, Videos. Günstig zu betreiben. Problem: Wenn du etwas suchst, musst du erst mal sortieren. Kann chaotisch werden (Data Swamp).

Details in der Übersicht

Merkmal	Data Warehouse	Big Data	Data Lake
Was ist es?	Speichersystem für strukturierte Business-Daten	Konzept/Herausforderung für riesige Datenmengen	Speichersystem für Rohdaten aller Art
Schema	Schema-on-Write (vordefiniert)	Variabel, je nach Technologie	Schema-on-Read (beim Auslesen definieren)
Datenqualität	Hoch – Daten werden bereinigt (ETL)	Variabel – kann auch "schmutzige" Daten enthalten	Niedrig bis mittel – Rohdaten, unverarbeitet
Nutzer	Business Analysten, Manager	Data Scientists, Entwickler, Analysten	Data Scientists, Data Engineers
Hauptzweck	Business Intelligence, Reporting	Verschiedene Anwendungsfälle: Machine Learning, Analysen, Prognosen	Flexibler Datenspeicher für zukünftige Nutzung
Kosten	Hoch (Speicher + Verarbeitung teuer)	Variabel (je nach Infrastruktur)	Niedrig (günstiger Cloud-Speicher)

Merkmal	Data Warehouse	Big Data	Data Lake
Datentyp	nur strukturierte Daten (Tabellen)	strukturiert, semistrukturiert, unstrukturiert	alle Typen: strukturiert, semi- und unstrukturiert
Flexibilität	niedrig – Schema muss vorher definiert werden	hoch – verschiedene Technologien nutzbar	sehr hoch – alles speichern, später entscheiden
Verarbeitungszeit	schnell bei definierten Abfragen	kann sehr langsam bei sehr großen Mengen sein	variabel – hängt davon ab, was man ausliest
Beispiel-Technologie	Snowflake, Amazon, Redshift, Google BigQuery, Teradata	Hadoop, Apache Spark, Apache Kafka, Flink	Amazon S3, Azure Data Lake, Google Cloud Storage, Databricks
Datenvolumen	GB bis TB (typisch)	TB bis PB und mehr	TB bis PB und mehr
Typische Frage	„Wie war unser Umsatz letztes Quartal?“	„Wie analysieren wir 10 Milliarden Events pro Tag?“	„Lass uns alle Daten speichern – vielleicht brauchen wir sie später.“

Data Warehouse

Einzelhandels-Reporting

Ein Supermarkt speichert alle Verkäufe, Lagerbestände und Kundendaten in einem Data Warehouse. Manager können schnell Berichte erstellen: "Top 10 Produkte", "Umsatz pro Filiale", "Quartalsvergleiche".

Banken-Finanzbericht

Eine Bank konsolidiert Daten aus allen Filialen und Produkten in einem Data Warehouse. Für Compliance-Berichte und Jahresabschlüsse sind die Daten sofort abrufbar – sauber und strukturiert.

Big Data

Social Media Analyse

Twitter verarbeitet Millionen Tweets pro Minute. Traditionelle Datenbanken können das nicht bewältigen. Big Data-Technologien (Hadoop, Spark) analysieren Trends in Echtzeit über Milliarden von Tweets.

Netflix Empfehlungen

Netflix analysiert täglich Milliarden Interaktionen: Was schauen Nutzer? Wann pausieren sie? Was bewerten sie? Big Data-Algorithmen nutzen diese Daten für personalisierte Empfehlungen.

Data Lake

IoT-Sensor-Daten

Eine Fabrik hat tausende Sensoren, die Temperatur, Druck, Vibration messen. Alle Rohdaten werden in einem Data Lake gespeichert. Data Scientists können später entscheiden, welche Analysen sinnvoll sind.

Gesundheitsdaten-Forschung

Ein Krankenhaus speichert alle Patientendaten (Röntgenbilder, Laborwerte, genetische Daten) in einem Data Lake. Forscher können später verschiedene Analysen durchführen, ohne alles neu zu strukturieren.

Was kann zusammenfassend gesagt werden?

Data Warehouse

ist wie eine gut organisierte Bibliothek – perfekt für Business Intelligence und Berichte, aber nur für strukturierte Daten.

Big Data

beschreibt die Herausforderung, mit riesigen, schnell wachsenden Datenmengen umzugehen – es braucht spezielle Tools und Infrastruktur.

Data Lake

ist ein flexibler, günstiger Speicher für alle Arten von Rohdaten – ideal, wenn man noch nicht genau weiß, was man analysieren will.

In der Praxis werden oft alle drei kombiniert:

- Daten landen erst im Data Lake, werden dann für Big Data-Analysen verwendet, und die wichtigsten Erkenntnisse fließen in ein Data Warehouse für regelmäßige Business-Berichte.
- Diese Architektur nennt man "Modern Data Stack" oder "Data Lakehouse" (Kombination aus Lake + Warehouse).

ETL – Extract, Transform, Load

Worum geht's?

ETL beschreibt einen dreistufigen Prozess, mit dem Daten aus einer oder mehreren Quellen in ein Zielsystem – meist ein Data Warehouse – überführt werden.

Extract (Extract)

Daten werden aus verschiedenen Quellsystemen gelesen. Das können sein: relationale Datenbanken, APIs, CSV-Dateien, ERP-Systeme, Log-Dateien usw.

Ziel: Rohdaten sammeln, ohne die Quellsysteme zu verändern.

Transform (Transformieren)

Die extrahierten Daten werden bereinigt, vereinheitlicht und in das gewünschte Format gebracht.

Typische Schritte:

- Dubletten entfernen
- Datumsformate vereinheitlichen (z. B. 01.01.2024 → 2024-01-01)
- Fehlende Werte behandeln
- Daten aus mehreren Quellen zusammenführen (Joining)
- Berechnungen durchführen (z. B. Umsatz = Menge × Preis)

Ziel: Saubere, konsistente und auswertbare Daten erzeugen.

Load (Laden)

Die transformierten Daten werden in das Zielsystem geschrieben – typischerweise ein Data Warehouse wie Snowflake, BigQuery oder ein klassisches SQL-System.

Ziel: Daten dauerhaft und abfragebereit speichern.

ETL - Analogien

Stell dir vor, du führst drei Lagerhäuser mit unterschiedlich sortierten Beständen zusammen:

Extract (Extract)

Du holst alle Kartons aus den drei Lagern.

Transform (Transformieren)

Du öffnest sie, sortierst den Inhalt, beschriftest alles einheitlich und schmeißt Defektes weg.

Load (Laden)

Du räumst alles ordentlich ins neue Zentrallager ein.

Strukturierte, semi-strukturierte und unstrukturierte Daten

Worum geht's?

Diese drei Kategorien beschreiben, wie organisiert und vorhersehbar das Format von Daten ist.

Daten folgen einem festen Schema – jeder Datensatz hat dieselben Felder, dieselben Datentypen und eine klare Struktur.

Typische Speicherorte: Relationale Datenbanken (mariaDB, PostgreSQL), Excel-Tabellen

Kundentabelle			
ID	Vorname	Nachname	Alter
1	Karl	Dall	47
2	Walter	Zabel	26

Vorteile am Beispiel obenstehender Tabelle:

- Leicht abfragbar (SQL), gut für Analysen, effizient gespeichert.

Nachteile am Beispiel obenstehender Tabelle:

- Unflexibel – ändert sich die Struktur, muss das Schema angepasst werden.

Daten haben keine starre Tabellenstruktur, bringen aber eine eigene interne Organisation mit – z. B. durch Tags oder Schlüssel-Wert-Paare (key-value pairs). Nicht jeder Datensatz muss dieselben Felder haben.

Typische Formate: JSON, XML, YAML, CSV (bedingt)

Die folgenden Datensätze sind im JSON-Format, haben aber unterschiedliche Felder – das wäre in einer klassischen Tabelle nicht möglich.

```
{ "id": 1, "name": „Karl Dall“, "alter": 47 }
```

```
{ "id": 2, „vorname“: „Walter“, „nachname“: „Zabel“, "hobbys": ["Lesen", "Sport"] }
```

Vorteile eines JSON-Formats:

- Flexibel, weit verbreitet im Web (APIs), gut lesbar

Nachteile eines JSON-Formats:

- Abfragen komplexer als bei strukturierten Daten, Qualität variiert.

Daten haben kein vordefiniertes Format. Es gibt keine Felder, keine Schlüssel – nur roher Inhalt.

Typische Formate: Texte, E-Mails, Bilder, Videos, Audio, PDFs, Social-Media-Posts

Beispiel: Eine E-Mail, ein Kundenbewertungstext, ein Foto, eine Sprachaufnahme

Vorteile:

- Sehr reichhaltig an Information, spiegelt die reale Welt wider

Nachteile:

- Schwer maschinell auszuwerten – braucht oft KI (Künstliche Intelligenz) / NLP (Natural Language Processing), um nutzbar zu werden.

Strukturierte, semistrukturierte und unstrukturierte Daten im detaillierten Vergleich

	strukturiert	semistrukturiert	unstrukturiert
Schema	Fest vorgegeben	Flexibel, selbstbeschreibend	Keines
Beispiele	SQL-Tabelle, Excel	JSON, XML, YAML	Bilder, Texte, Videos
Abfragen	SQL	XPath, JSONPath, NoSQL	KI, NLP, manuelle Analyse
Speicher	Relationale DB	NoSQL-DB, DataLake	Object Storage, Data Lake

Schätzungen zufolge sind etwa 80 % aller weltweit vorhandenen Daten unstrukturiert – weshalb Technologien wie maschinelles Lernen und NLP* heute so wichtig sind, um diesen riesigen Datenschatz überhaupt nutzbar zu machen.

* NLP steht für Natural Language Processing (deutsch: Verarbeitung natürlicher Sprache) und ist ein Teilgebiet der künstlichen Intelligenz. Es beschäftigt sich damit, wie Computer menschliche Sprache – geschrieben oder gesprochen – verstehen, interpretieren und erzeugen können.



Vielen Dank für eure Aufmerksamkeit.